

随机信号智能体配置教程

版本复现与部署说明

作者：YangCheng

日期：2026 年 6 月

目录

1	项目概述	2
2	软件架构	2
2.1	目录结构	2
2.2	运行流程	3
3	本地复现	4
3.1	环境要求	4
3.2	安装依赖	4
3.3	本地启动	4
4	外部 API 配置	4
4.1	配置文件	4
4.2	配置字段	5
4.3	使用配置启动	5
5	云服务器部署	6
5.1	服务器环境	6
5.2	上传代码	6
5.3	创建 Python 环境	6
5.4	填写服务器端配置	6
5.5	直接启动	7
5.6	systemd 后台托管	7
5.7	HTTPS 与麦克风权限	7
6	Docker 部署	8
7	验证方法	8
7.1	健康检查	8
7.2	对话测试	8
7.3	文件上传测试	9
8	故障排查	9
8.1	服务启动后外部模型不可用	9
8.2	云端页面无法访问	9
8.3	麦克风无法授权	9

1 项目概述

随机信号智能体是面向“随机信号”课程的可复现实验系统。系统以 Web 对话界面为入口，后端接收自然语言任务后调用本地随机信号处理工具，完成采集、预处理、时域分析、频域分析、随机过程分析和结果解释。

发布版以压缩包形式交付，文件名为 随机信号智能体.zip。复现者下载后先解压压缩包，再进入解压后的项目根目录执行下列命令。本文档中的“项目根目录”均指解压后包含 server.py、requirements.txt、src、web、config 等文件和文件夹的目录。

```
随机信号智能体/  
server.py  
requirements.txt  
config/  
scripts/  
src/  
web/
```

当前发布版包含以下功能：

- 仿真随机信号采集：支持采样率、时长、主频、噪声强度和随机种子参数。
- 文件上传分析：支持单列采样值或“时间, 采样值”双列 CSV/TXT 文件。
- 麦克风网关：浏览器端采集音频后上传至后端，完成音频降噪、时域分析和频域分析。
- 预处理工具链：包含滑动平均、中值滤波、鲁棒异常点抑制和去均值处理。
- 时域分析：输出均值、方差、标准差、RMS、峰峰值、零交叉率和自相关指标。
- 频域分析：输出功率谱、主频、谱质心、谱带宽和谱熵指标。
- Agent 模式：自动比较多种预处理方法，输出工具调用轨迹、风险等级和控制指令。
- 外部模型增强：支持 OpenAI 兼容 Chat Completions API；未填写 API 信息时，系统使用本地规则链完成分析。

2 软件架构

发布版采用“前端控制台 + Python HTTP 后端 + 本地算法工具链 + 外部模型增强通道”的结构。外部模型通道只承担任务规划和自然语言解释，本地算法工具负责实际信号采集、预处理和特征计算。

2.1 目录结构

路径	说明
<code>server.py</code>	HTTP/SSE 后端入口，提供首页、对话、流式对话、上传和健康检查接口。
<code>web/chat.html</code>	智能体控制台前端，包含聊天区、波形图、频谱图、工具库和麦克风采集界面。
<code>src/dialogue_agent.py</code>	对话智能体核心，负责意图解析、会话状态、工具调度、Agent 模式和回复生成。
<code>src/llm_client.py</code>	OpenAI 兼容 API 客户端，读取环境变量后调用外部模型。
<code>src/acquisition.py</code>	信号采集模块，支持仿真、文件、实时流式和麦克风采集通道。
<code>src/preprocessing.py</code>	预处理算法模块，包括异常点检测、去均值、平滑和中值滤波。
<code>src/analysis.py</code>	基础时域与频域特征分析模块。
<code>src/advanced_analysis.py</code>	随机过程综合分析模块。
<code>src/signal_processing.py</code>	随机信号生成、功率谱、自相关和数据降采样等底层函数。
<code>config/server.env.example</code>	服务配置模板，发布版不包含真实 API Key 或云服务器地址。
<code>scripts/start_server.ps1</code>	Windows 启动脚本，读取 <code>config/server.env</code> 。
<code>scripts/start_server.sh</code>	Linux/macOS 启动脚本，读取 <code>config/server.env</code> 。
<code>scripts/random-signal-agent.service.example</code>	systemd 服务模板，云端部署时替换为复现者的用户和路径。

2.2 运行流程

1. 复现者在浏览器中打开控制台，输入分析请求或上传信号文件。
2. 前端通过 HTTP 或 SSE 请求发送任务到 `server.py`。
3. `server.py` 将请求交给 `RandomSignalDialogueAgent`。
4. 智能体读取会话状态、Agent 模式和工具库参数。
5. 外部模型 API 已配置时，智能体调用模型生成工具调度方案；外部模型 API 未配置或调用失败时，智能体使用本地规则链完成调度。
6. 本地工具执行采集、预处理、时域分析、频域分析和随机过程分析。

7. 后端将结果序列化为 JSON，前端刷新波形、频谱、指标卡片和文字解释。

3 本地复现

3.1 环境要求

- Python 3.10 或更高版本。
- pip。
- Edge、Chrome 或 Firefox 浏览器。
- Python 依赖：numpy。

3.2 安装依赖

在 PowerShell 中进入解压后的项目根目录并安装依赖。下列路径中的 随机信号智能体 为压缩包解压后得到的项目文件夹名称：

```
cd ".\随机信号智能体"  
pip install -r requirements.txt
```

3.3 本地启动

未配置外部 API 时，系统使用本地规则工具链运行：

```
python .\server.py --host 127.0.0.1 --port 8000
```

浏览器访问：

```
http://127.0.0.1:8000
```

测试请求如下：

```
采集一段 8 秒、采样率 200Hz、主频 8Hz 的随机信号，并完成时域和频域分析。
```

4 外部 API 配置

4.1 配置文件

外部 API 信息写入解压后项目根目录中的 config/server.env 文件。发布包只提供配置模板 config/server.env.example，未包含真实 API Key、供应商账号或私有 endpoint。第一次配置时，先复制模板并创建私有配置文件：

```
Copy-Item .\config\server.env.example .\config\server.env  
notepad .\config\server.env
```

Linux/macOS 命令如下：

```
cp config/server.env.example config/server.env
```

```
nano config/server.env
```

4.2 配置字段

以下内容在 `config/server.env` 文件中修改。字段等号右侧为空的位置由复现者填写自己的外部模型 API 信息：

```
RS_AGENT_HOST=0.0.0.0
RS_AGENT_PORT=8000

# 在此处填写外部模型 API Base URL，例如供应商控制台给出的 OpenAI 兼容地址。
RS_AGENT_LLM_BASE_URL=

# 在此处填写模型名称，例如供应商文档中标注的 chat/completions 模型名。
RS_AGENT_LLM_MODEL=

# 在此处填写自己的 API Key。公开发布时不写入真实 Key。
RS_AGENT_LLM_API_KEY=

RS_AGENT_LLM_TIMEOUT=45
RS_AGENT_LOG_FILE=logs/server/server.log
```

字段	含义
RS_AGENT_HOST	服务监听地址。本地测试使用 127.0.0.1；云服务器对外访问使用 0.0.0.0。
RS_AGENT_PORT	服务端口，默认值为 8000。云端安全组和防火墙放行同一端口。
RS_AGENT_LLM_BASE_URL	外部模型 API Base URL，填写 OpenAI 兼容接口地址。
RS_AGENT_LLM_MODEL	模型名称，按供应商文档填写。
RS_AGENT_LLM_API_KEY	API Key，只保存在本机或服务器的私有配置文件中。
RS_AGENT_LLM_TIMEOUT	外部模型请求超时时间，单位为秒。
RS_AGENT_LOG_FILE	启动脚本输出日志的位置。

4.3 使用配置启动

Windows 启动命令：

```
.\scripts\start_server.ps1
```

Linux/macOS 启动命令：

```
bash scripts/start_server.sh
```

健康检查命令如下：

```
Invoke-WebRequest http://127.0.0.1:8000/api/health
```

返回结果中相关字段含义如下：

- `llm.configured=true`：外部模型配置已被服务读取。
- `llm.configured=false`：系统正在使用本地规则链。

5 云服务器部署

5.1 服务器环境

云端复现环境采用 Linux 服务器，包含以下组件：

- Ubuntu 22.04/24.04 或等价 Linux 发行版。
- Python 3.10 或更高版本。
- 对外开放的 TCP 端口，本文档使用 8000。
- 公网麦克风采集功能使用 HTTPS 或 localhost 安全上下文。

发布包未包含固定公网 IP、服务器用户名或私有目录。以下命令中的路径和用户名均为占位符，复现时替换为实际服务器环境。

5.2 上传代码

将“代码”目录上传到服务器目录：

```
/path/to/random-signal-agent
```

该路径为占位路径，不是项目内置路径。

5.3 创建 Python 环境

```
cd /path/to/random-signal-agent
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

5.4 填写服务器端配置

```
cp config/server.env.example config/server.env
nano config/server.env
```

云端配置内容如下：

```
RS_AGENT_HOST=0.0.0.0
RS_AGENT_PORT=8000
RS_AGENT_LLM_BASE_URL=在此处填写自己的外部模型 API Base URL
RS_AGENT_LLM_MODEL=在此处填写自己的模型名称
RS_AGENT_LLM_API_KEY=在此处填写自己的 API Key
```

5.5 直接启动

```
bash scripts/start_server.sh
```

云平台控制台放行 TCP 8000 端口后，访问地址为：

```
http://你的服务器公网IP:8000
```

5.6 systemd 后台托管

发布包提供 systemd 服务模板：

```
scripts/random-signal-agent.service.example
```

复制到 systemd 目录：

```
sudo cp scripts/random-signal-agent.service.example /etc/systemd/system/random-signal-agent.service
sudo nano /etc/systemd/system/random-signal-agent.service
```

将模板中的占位符替换为服务器实际信息：

```
User=YOUR_LINUX_USER
WorkingDirectory=/path/to/random-signal-agent
EnvironmentFile=/path/to/random-signal-agent/config/server.env
ExecStart=/usr/bin/python3 /path/to/random-signal-agent/server.py --host 0.0.0.0 --port 8000
```

启动服务：

```
sudo systemctl daemon-reload
sudo systemctl enable --now random-signal-agent.service
sudo systemctl status random-signal-agent.service
```

查看日志：

```
journalctl -u random-signal-agent.service -n 100 --no-pager
```

5.7 HTTPS 与麦克风权限

浏览器麦克风权限依赖安全上下文。公网部署中，麦克风采集页面使用 HTTPS 地址；本地复现中，http://127.0.0.1:8000 满足 localhost 安全上下文。公网 HTTPS 部署由反向代理完成，典型拓扑为：

浏览器 HTTPS 请求 -> Nginx/Caddy -> 127.0.0.1:8000 -> Python 后端

6 Docker 部署

复制 Docker 环境变量模板：

```
cp .env.example .env
nano .env
```

填写外部模型配置：

```
RS_AGENT_LLM_BASE_URL=在此处填写自己的外部模型 API Base URL
RS_AGENT_LLM_MODEL=在此处填写自己的模型名称
RS_AGENT_LLM_API_KEY=在此处填写自己的 API Key
```

启动容器：

```
docker compose up -d --build
```

访问地址：

```
http://你的服务器公网IP:8000
```

7 验证方法

7.1 健康检查

健康检查接口如下：

```
GET /api/health
```

本地执行命令：

```
Invoke-WebRequest http://127.0.0.1:8000/api/health
```

检查项如下：

- status 返回 ok。
- llm.configured=true 表示外部模型 API 已配置。
- llm.configured=false 表示系统使用本地规则链。

7.2 对话测试

测试输入如下：

```
采集一段 10 秒采样率 200Hz 的随机信号，比较滑动平均和中值滤波，然后输出时域频域分析。
```

系统输出包含以下结果：

- 原始信号和预处理信号波形。

- 主频和功率谱图。
- 均值、方差、RMS、SNR、谱熵等指标。
- Agent 模式工具调用时间线和控制指令。

7.3 文件上传测试

CSV 单列格式如下：

```
0.12  
0.48  
-0.16
```

CSV 双列格式如下：

```
0.000,0.12  
0.005,0.48  
0.010,-0.16
```

上传后填写采样率，系统按上传数据计算时域和频域特征。

8 故障排查

8.1 服务启动后外部模型不可用

检查 `config/server.env`：

- `RS_AGENT_LLM_BASE_URL` 已填写供应商给出的 OpenAI 兼容 Base URL。
- `RS_AGENT_LLM_MODEL` 已填写供应商支持的模型名。
- `RS_AGENT_LLM_API_KEY` 已填写有效 Key。
- 服务器网络能够访问外部 API。

8.2 云端页面无法访问

- Python 服务监听地址为 `0.0.0.0:8000`。
- 云平台安全组已放行 TCP 8000。
- 服务器本机防火墙已放行 TCP 8000。
- 服务器内部执行 `curl http://127.0.0.1:8000/api/health` 返回正常结果。

8.3 麦克风无法授权

- 本地复现地址为 `http://127.0.0.1:8000`。
- 公网复现地址为 HTTPS 地址。

- 课程演示场景中，浏览器可临时设置为 HTTP 麦克风演示模式。Windows Edge 示例命令如下，将其中的 `http://你的服务器公网 IP:8000` 替换为实际访问地址：

```
msedge.exe --user-data-dir="%TEMP%\rs-agent-edge-http-mic" --unsafely-treat-  
insecure-origin-as-secure="http://你的服务器公网IP:8000" --new-window "http://  
你的服务器公网IP:8000"
```

- 浏览器站点权限中麦克风权限为允许状态。